

# SSH BRUTE FORCE ATTACK

## Detection & Analysis Report

Using Wazuh SIEM | MITRE ATT&CK Mapped | Python Integration in Progress

| Analyst | Target System             | Attack Type     | Date           |
|---------|---------------------------|-----------------|----------------|
| Yessy   | KALI-UTM<br>(10.11.3.106) | SSH Brute Force | April 21, 2026 |

| MITRE Tactic      | MITRE Technique     | Wazuh Rule      | Tool Used     |
|-------------------|---------------------|-----------------|---------------|
| Credential Access | T1110 - Brute Force | 5712 (Level 10) | Wazuh v4.14.4 |

## Executive Summary

This report documents a hands-on cybersecurity lab exercise in which an SSH brute force attack was simulated against a Kali Linux virtual machine monitored by the Wazuh SIEM platform. The goal was to demonstrate the full attack detection lifecycle: enabling the attack surface, understanding detection rules, executing the attack, observing alerts in real time, and documenting findings.

Wazuh successfully detected the brute force activity and fired Rule 5712 (Level 10), confirming the platform's effectiveness as a real-time threat detection tool. All findings are mapped to the MITRE ATT&CK framework.

*Project Update: This lab is part of an ongoing cybersecurity project that is now beginning to incorporate Python scripting. Python is being introduced to automate analysis tasks, parse Wazuh logs programmatically, and build custom tooling around the SIEM environment. This marks an exciting step forward in the project's technical depth.*

## MITRE ATT&CK Framework Mapping

This attack maps to the following MITRE ATT&CK technique:

---

| Field                | Details                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
| Tactic               | Credential Access (TA0006)                                                                                                 |
| Technique            | T1110 – Brute Force                                                                                                        |
| Sub-technique        | T1110.001 – Password Guessing                                                                                              |
| Description          | Adversaries attempt to gain access to accounts by systematically guessing passwords using non-existent usernames over SSH. |
| Platform             | Linux (Kali GNU/Linux 2023.3)                                                                                              |
| Protocol Targeted    | SSH (Port 22)                                                                                                              |
| Wazuh Rule Triggered | 5712 – sshd: brute force trying to get access to the system. Non-existent user.                                            |
| Rule Level           | 10 (Medium-High severity)                                                                                                  |
| Compliance Tags      | PCI DSS 11.4, 10.2.4, 10.2.5   HIPAA 164.312.b   NIST-800-53 SI.4, AU.14, AC.7                                             |

# Step-by-Step Attack Walkthrough

---

## STEP 1 Activate SSH on Kali Linux

---

Before any attack can be simulated or detected, the target machine needs to be listening for SSH connections. SSH was enabled on the Kali Linux virtual machine running inside UTM on an Apple Silicon Mac.

---

---

### Commands executed on Kali Linux:

---

```
sudo systemctl start ssh
```

---

```
sudo systemctl status ssh
```

---

```
ifconfig
```

---

---

### What we confirmed:

---

- SSH service status: active (running) since Tue 2026-04-21 12:10:54 PDT
- Server listening on 0.0.0.0 port 22 (all interfaces)
- Kali IP address confirmed: 10.11.3.106

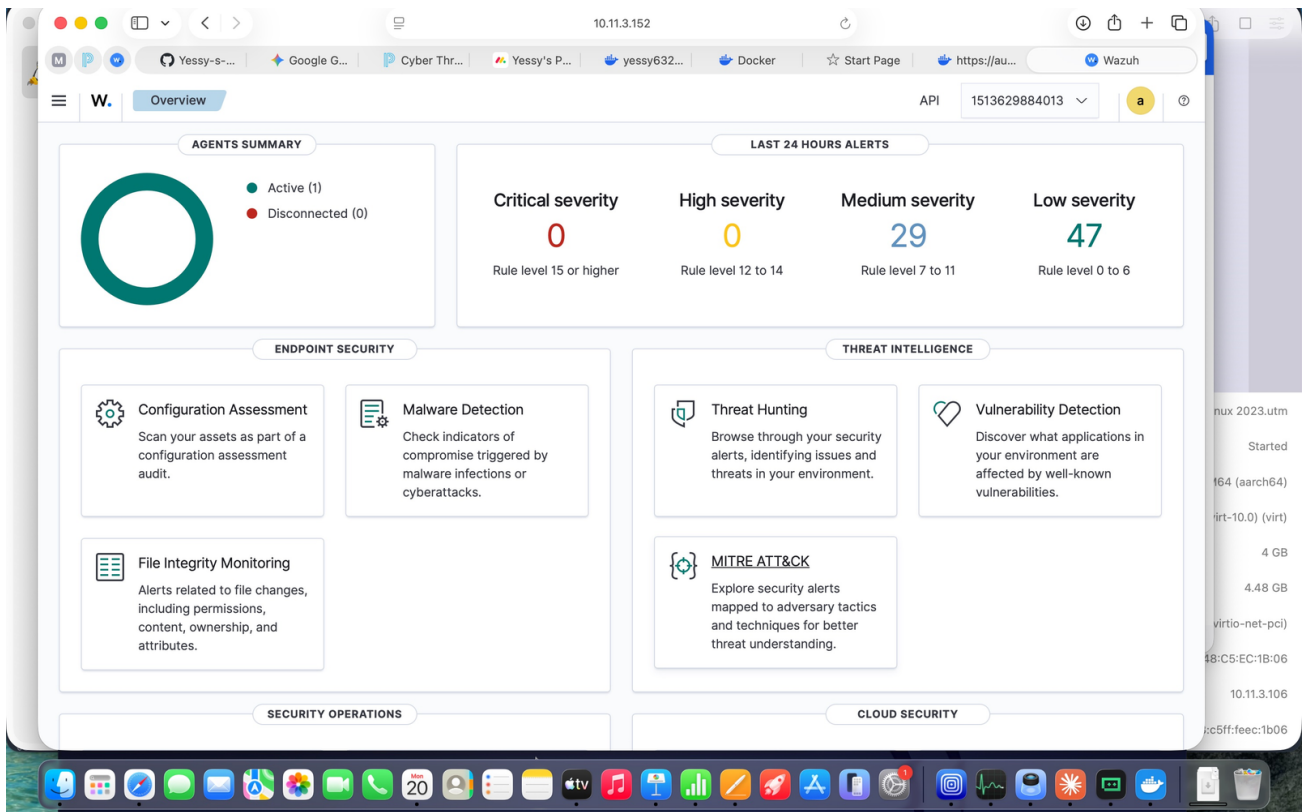


Figure 1 – Wazuh Dashboard Overview showing 1 active agent (KALI-UTM) and 29 medium + 47 low severity alerts in the last 24 hours

Rules (4,513)

From here you can manage your rules.

Search

| ID ↑ | Description                                    |
|------|------------------------------------------------|
| 1    | Generic template for all syslog rules.         |
| 2    | Generic template for all firewall rules.       |
| 3    | Generic template for all ids rules.            |
| 4    | Generic template for all web rules.            |
| 5    | Generic template for all web proxy rules.      |
| 6    | Generic template for all windows rules.        |
| 7    | Generic template for all wazuh rules.          |
| 200  | Grouping of wazuh rules.                       |
| 201  | Agent event queue rule                         |
| 202  | Agent event queue is level full.               |
| 203  | Agent event queue is full. Events may be lost. |
| 204  | Agent event queue is flooded. Check the agent  |
| 205  | Agent event queue is back to normal load.      |
| 210  | Remote upgrade alert                           |
| 211  | Remote installation alert                      |
| 212  | Remote upgrade started                         |

ssh: brute force trying to get access to the system. Non existent user.

Information

|                                       |       |                     |               |
|---------------------------------------|-------|---------------------|---------------|
| ID                                    | Level | File                | Path          |
| 5712                                  | 10    | 0095-sshd_rules.xml | ruleset/rules |
| Groups                                |       |                     |               |
| authentication_failures, syslog, sshd |       |                     |               |

Details

|                |           |        |                |
|----------------|-----------|--------|----------------|
| Frequency      | Timeframe | Ignore | If_matched_sid |
| 8              | 120       | 60     | 5710           |
| Same_source_ip |           |        |                |
| true           |           |        |                |

Compliance

|                            |                    |                   |                   |
|----------------------------|--------------------|-------------------|-------------------|
| PCI DSS                    | GDPR               | HIPAA             | NIST-800-53       |
| 11.4, 10.2.4, 10.2.5       | IV_35.7.d, IV_32.2 | 164.312.b         | SI.4, AU.14, AC.7 |
| TSC                        | MITRE Techniques   | MITRE Tactics     |                   |
| CC6.1, CC6.8, CC7.2, CC7.3 | Brute Force        | Credential Access |                   |

Related rules

| ID ↑ | Description            | Groups       | Compliance | Level | File                |
|------|------------------------|--------------|------------|-------|---------------------|
| 5700 | SSHD messages grouped. | syslog, sshd |            | 0     | 0095-sshd_rules.xml |

Figure 2 – Kali Linux terminal showing SSH service active and running on port 22, with IP address 10.11.3.106 confirmed via ifconfig

### Why this matters:

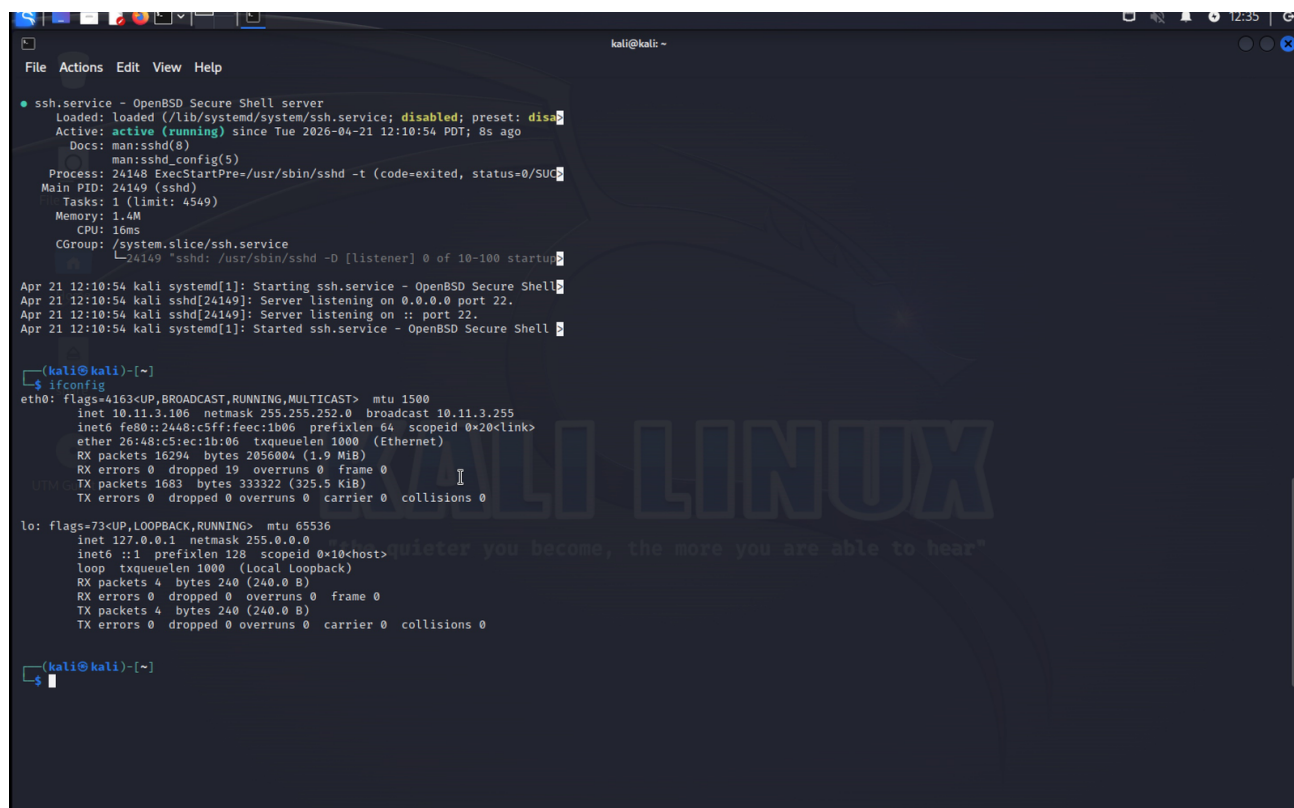
SSH is one of the most commonly targeted services in real-world attacks. Automated bots constantly scan the internet for open SSH ports on port 22. By enabling it, we created the attack surface that Wazuh would monitor.

## STEP 2 Review Wazuh Detection Rule 5712

Before running the attack, we studied the Wazuh rule that would detect it. Understanding the rule helps us know the threshold that triggers an alert and what evidence Wazuh collects.

### Rule 5712 – Key Details:

|                 |                                                                          |
|-----------------|--------------------------------------------------------------------------|
| Rule ID         | 5712                                                                     |
| Rule Level      | 10 (Medium-High – triggers as medium alert in Wazuh)                     |
| Description     | sshd: brute force trying to get access to the system. Non-existent user. |
| File            | 0095-sshd_rules.xml                                                      |
| Groups          | authentication_failures, syslog, sshd                                    |
| Frequency       | 8 attempts within 120 seconds from same IP                               |
| Same Source IP  | true                                                                     |
| Parent Rule     | 5710 – sshd: Attempt to login using a non-existent user                  |
| MITRE Technique | T1110 – Brute Force                                                      |
| MITRE Tactic    | Credential Access                                                        |



```
kali@kali: ~  
File Actions Edit View Help  
● ssh.service - OpenBSD Secure Shell server  
Loaded: loaded (/lib/systemd/system/ssh.service; disabled; preset: disabled)  
Active: active (running) since Tue 2026-04-21 12:10:54 PDT; 8s ago  
Docs: man:sshd(8)  
man:sshd_config(5)  
Process: 24148 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)  
Main PID: 24149 (sshd)  
Tasks: 1 (limit: 4549)  
Memory: 1.4M  
CPU: 16ms  
CGroup: /system.slice/ssh.service  
└─24149 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"  
  
Apr 21 12:10:54 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell:  
Apr 21 12:10:54 kali sshd[24149]: Server listening on 0.0.0.0 port 22.  
Apr 21 12:10:54 kali sshd[24149]: Server listening on :: port 22.  
Apr 21 12:10:54 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell:  
  
(kali@kali)~  
$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 10.11.3.106 netmask 255.255.252.0 broadcast 10.11.3.255  
    inet6 fe80::2448:c5ff:feec:1b06 prefixlen 64 scopeid 0x20<link>  
    ether 26:48:c5:ec:1b:06 txqueuelen 1000 (Ethernet)  
    RX packets 16294 bytes 2056004 (1.9 MiB)  
    RX errors 0 dropped 19 overruns 0 frame 0  
    TX packets 1683 bytes 333322 (325.5 KiB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 4 bytes 240 (240.0 B)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 4 bytes 240 (240.0 B)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
(kali@kali)~  
$
```

Figure 3 – Wazuh Rules panel showing Rule 5712 detail: Level 10, frequency of 8 attempts in 120 seconds, MITRE mapped to Credential Access / Brute Force

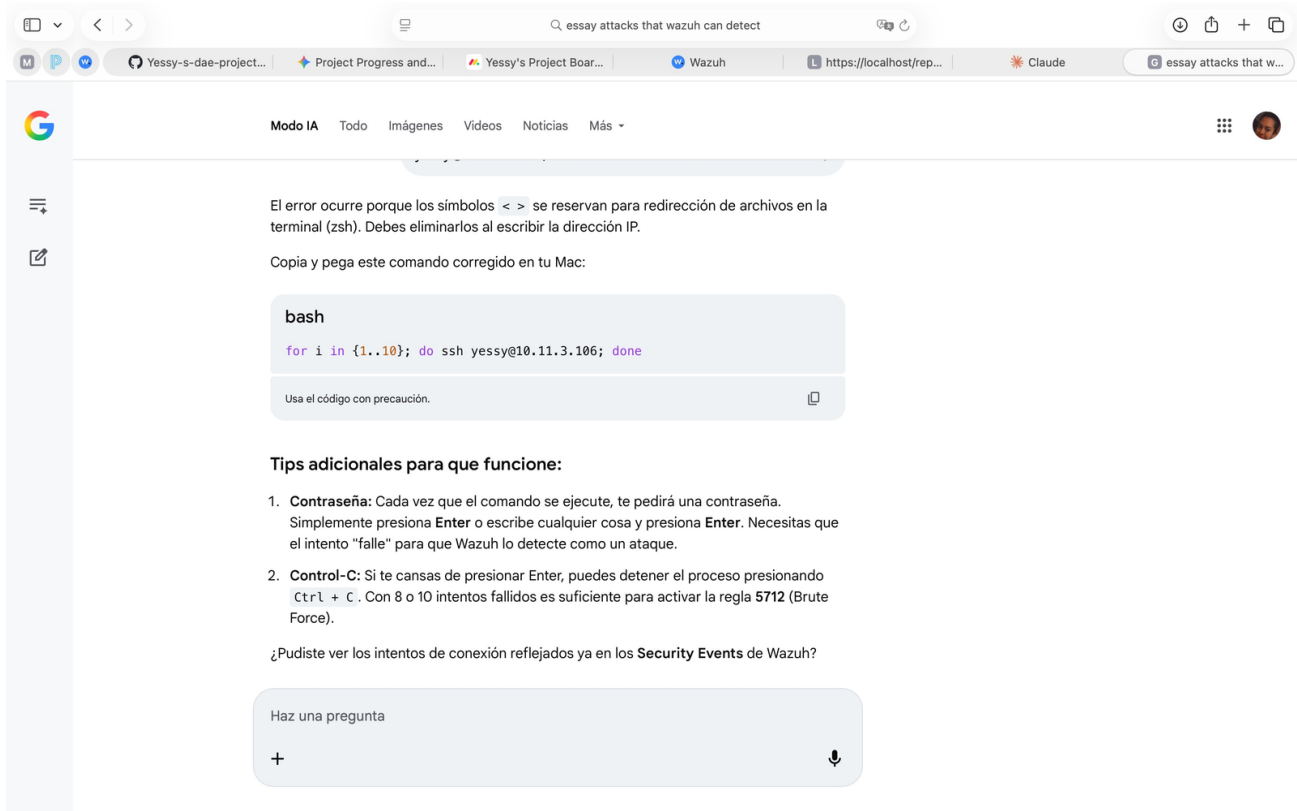


Figure 4 – The raw XML file (0095-sshd\_rules.xml) inside Wazuh showing the rule definitions for SSH including Rules 5700-5712

### In plain English:

Rule 5712 fires when the same IP address attempts to log in with a username that does not exist on the system, 8 or more times within 2 minutes. It builds on Rule 5710 (single failed attempt) – once 8 of those fire from the same source, the brute force alert triggers.

### Compliance tags triggered:

- PCI DSS 11.4, 10.2.4, 10.2.5 – monitoring and testing security controls
- HIPAA 164.312.b – audit controls for electronic systems
- NIST 800-53 SI.4, AU.14, AC.7 – system monitoring and account lockout

## STEP 3 Simulate SSH Brute Force Attack

With SSH running on Kali (10.11.3.106) and Wazuh monitoring the agent, the brute force attack was simulated from the Mac host. The goal: generate enough failed login attempts to cross Rule 5712's threshold.

### Attack command used:

```
for i in {1..10}; do ssh yessy@10.11.3.106; done
```

### What happened:

- The loop attempted to SSH into Kali 10 times using the username 'yessy'
- 'yessy' is a non-existent user on the Kali system
- Each attempt prompted for a password – wrong passwords were entered intentionally
- After each failure SSH returned: Permission denied, please try again
- After 3 failed tries per session: Permission denied (publickey, password)
- Connection closed by 10.11.3.106 port 22 – then the loop started the next attempt

```
--- 10.11.3.106 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 0.819/1.063/1.502/0.311 ms
adult@DAEDMAC1001 single-node % for i in {1..10}; do ssh yessy@10.11.3.106; done

zsh: parse error near `;'
adult@DAEDMAC1001 single-node % for i in {1..10}; do ssh yessy@10.11.3.106>
[
zsh: parse error near `\'
adult@DAEDMAC1001 single-node % for i in {1..10}; do ssh yessy@10.11.3.106; done

The authenticity of host '10.11.3.106 (10.11.3.106)' can't be established.
ED25519 key fingerprint is: SHA256:kXKpvi6WVUW+3YFe+Nm52wNLSxQr85ZpVq9IRZJxfiE
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.11.3.106' (ED25519) to the list of known hosts.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
yessy@10.11.3.106: Permission denied (publickey,password).
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
yessy@10.11.3.106: Permission denied (publickey,password).
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
yessy@10.11.3.106: Permission denied (publickey,password).
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
yessy@10.11.3.106: Permission denied (publickey,password).
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Permission denied, please try again.
yessy@10.11.3.106's password:
Connection closed by 10.11.3.106 port 22
yessy@10.11.3.106's password:
```

Figure 5 – Mac terminal showing the SSH brute force loop in action: repeated 'Permission denied' responses and connection closures from Kali (10.11.3.106)

*Note: The zsh shell on Mac initially gave a parse error with angle brackets in the command. The corrected bash-compatible syntax shown above was used after quick troubleshooting. This demonstrates real-world problem-solving during a lab exercise.*





# STEP 4 Observe Alerts in Wazuh

After running the brute force command, the Wazuh Threat Hunting module was used to review the security events. The KALI-UTM agent (ID: 001) showed a clear spike in authentication failure events.

## Alerts Summary Table:

| Rule ID | Description                                                | Level | Count | Severity     |
|---------|------------------------------------------------------------|-------|-------|--------------|
| 5710    | sshd: Attempt to login using a non-existent user           | 5     | 17    | Low-Med      |
| 5503    | PAM: User login failed.                                    | 5     | 5     | Low-Med      |
| 2502    | syslog: User missed the password more than one time        | 10    | 4     | Medium       |
| 5712    | sshd: brute force trying to get access. Non-existent user. | 10    | 2     | ⚠️<br>Medium |

Total: 28 alerts | 0 successful logins | 0 critical (Level 12+) events

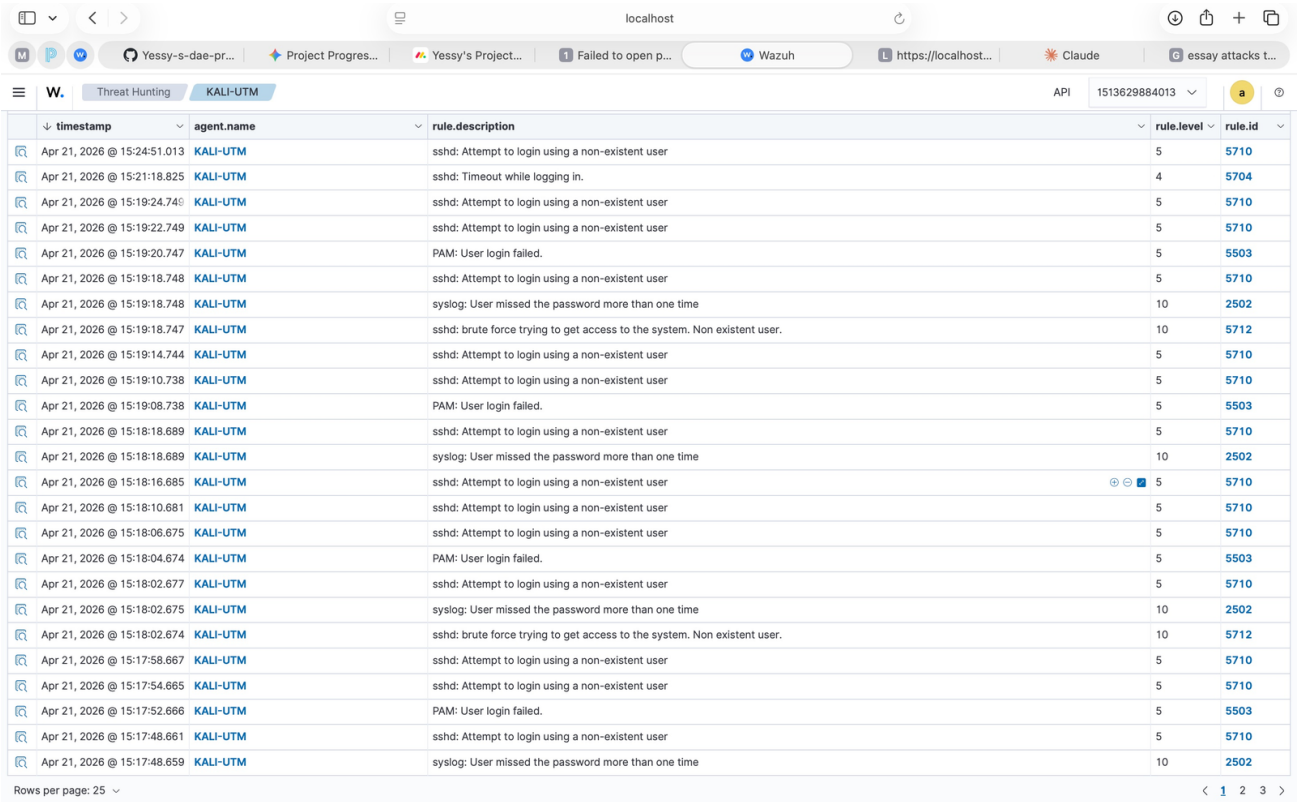


Figure 6 – Wazuh Threat Hunting Events view showing 53 total hits for agent KALI-UTM with a clear spike at 15:00 – exactly when the brute force ran

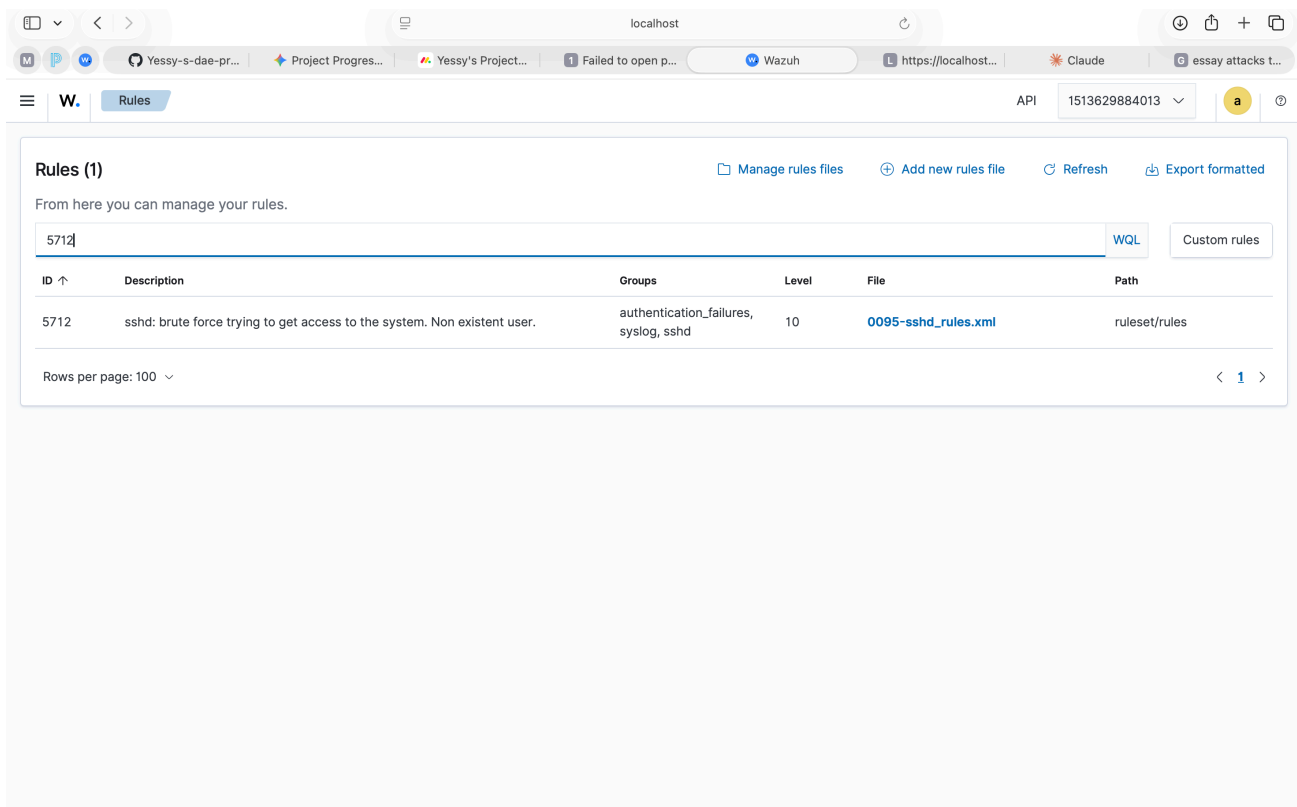


Figure 7 – Detailed alert list in Wazuh showing Rule 5712 (brute force) and Rule 5710 (individual attempts) firing in sequence from KALI-UTM

### What the alerts confirmed:

- Multiple Rule 5710 events – each individual failed SSH login attempt captured
- Rule 5712 fired twice – confirming the brute force threshold was crossed
- Rule 2502 correlated the repeated wrong-password pattern via syslog
- All events tied to agent KALI-UTM (ID: 001) at IP 10.11.3.106

## STEP 5 Document the Process – Findings & Lessons Learned

This final step captures the key findings, what was proven, and the security lessons demonstrated through this exercise.

### What Was Proven

- Wazuh successfully monitors SSH authentication in real time
- Rule 5712 correctly triggered after 8+ failed login attempts within 2 minutes
- The alert included full context: source agent, timestamp, and rule ID
- The Threat Hunting module made it easy to trace the full attack timeline
- The agent (KALI-UTM) was properly enrolled and reporting to the Wazuh manager

### Environment Used

|                      |                                                                                      |
|----------------------|--------------------------------------------------------------------------------------|
| <b>Wazuh Server</b>  | Docker single-node deployment (manager, indexer, dashboard) v4.14.4                  |
| <b>Wazuh Agent</b>   | KALI-UTM – Agent ID 001 on Kali Linux 2023.3 (ARM64)                                 |
| <b>Attacker Host</b> | Mac (Apple Silicon) running UTM/QEMU virtual machine                                 |
| <b>Network</b>       | 10.11.3.0/22 subnet – host-only between Mac and VM                                   |
| <b>Attack Tool</b>   | Native OpenSSH client via shell loop (no special tools needed)                       |
| <b>Monitoring</b>    | Wazuh Dashboard at <a href="https://localhost">https://localhost</a> (Docker-hosted) |

### Security Lessons Demonstrated

- **Lesson 1:** SSH brute force is a real, common attack – bots scan the internet for open port 22 constantly
- **Lesson 2:** SIEM tools like Wazuh are essential – without it, failed logins would be invisible unless manually reviewing `/var/log/auth.log`
- **Lesson 3:** Rule tuning matters – the 120-second window and `same_source_ip` filter avoid false positives from normal users mistyping
- **Lesson 4:** Compliance is built-in – detecting brute force automatically satisfies PCI DSS, HIPAA, and NIST requirements
- **Lesson 5:** Defense recommendations: use SSH key-only auth, fail2ban, change default port, and enable Wazuh Active Response

### Python Integration – Next Steps

*The project is now beginning to incorporate Python. This is a natural and powerful progression – Python allows us to go beyond the Wazuh dashboard and interact with security data programmatically.*

### Python is being introduced to this project to enable:

- Log parsing – reading and analyzing /var/log/auth.log to extract failed login attempts programmatically
- Wazuh API integration – querying the Wazuh REST API with Python to pull alerts, agent status, and rule hits automatically
- Alert automation – writing scripts that trigger notifications or responses when specific rules like 5712 fire
- Custom reporting – generating summaries and statistics from raw Wazuh event data without manually clicking through the dashboard
- Future: building tools that correlate multiple attack indicators across agents and time windows

### Example Python direction (Wazuh API query):

```
import requests
# Authenticate and query Wazuh API for Rule 5712 alerts
response = requests.get('https://localhost:55000/alerts?rule.id=5712',
                        headers={'Authorization': 'Bearer <token>'},
                        verify=False)
print(response.json())
```

Adding Python to this project demonstrates a full-stack security engineering mindset: not just using tools, but building automation and custom tooling on top of them – a key skill in real SOC and security engineering roles.

### Recommended Mitigations (Real-World)

| Control                      | Description                                                             |
|------------------------------|-------------------------------------------------------------------------|
| <b>Disable Password Auth</b> | Require SSH key pairs only – eliminates password guessing entirely      |
| <b>Use fail2ban</b>          | Auto-blocks IPs after N failed attempts at the firewall level           |
| <b>Change SSH Port</b>       | Moving from port 22 to a non-standard port reduces automated scanning   |
| <b>Wazuh Active Response</b> | Configure Wazuh to auto-block attacker IPs when Rule 5712 fires         |
| <b>Python Alert Scripts</b>  | Use Python to build custom alerting on top of Wazuh for faster response |

## Conclusion

---

**This lab successfully demonstrated a complete attack detection cycle. Starting from a zero-access position, SSH was enabled on Kali Linux, Rule 5712 was studied and understood, a brute force attack was simulated, and the resulting alerts were captured and analyzed in Wazuh – all backed by real screenshots from the live environment.**

---

**The exercise proves practical competency in: endpoint security monitoring, SIEM rule analysis, threat hunting, attack simulation, and compliance framework mapping (MITRE ATT&CK, PCI DSS, HIPAA, NIST 800-53).**

---

***With Python now entering the picture, this project is leveling up from a tool-user perspective to a security engineer perspective – building automation, custom queries, and programmatic analysis on top of the Wazuh platform.***

---